

Personne 1 — Backend Node.js

Initialiser le projet Node.js :

npm init

npm install express mysql2 bcrypt jsonwebtoken

dotenv cors nodemon

Structure à créer :

backend/

├— config/

| └─ db.js → connexion MySQL

├— controllers/

| └─ authController.js

├— middlewares/

| └─ authMiddleware.js

├— routes/

| └─ authRoutes.js

├— services/

| └─ otpService.js

├— .env → variables d'environnement

└─ server.js → point d'entrée

Base de données & connexion

1. Écrire le script SQL de création des tables :

CREATE TABLE acheteur (...)

CREATE TABLE organisateur (...)

CREATE TABLE administrateur (...)

CREATE TABLE controleur (...)

CREATE TABLE codeOTP (...)

2. Configurer db.js :

- connexion MySQL avec mysql2
- tester la connexion au démarrage

3. Insérer des données de test :

- 1 organisateur de test
- 1 administrateur de test
- 1 acheteur de test

Routes OTP (acheteur)

POST /api/auth/otp/envoyer

- reçoit { numeroTel }
- génère code 6 chiffres
- stocke dans table codeOTP
(dateExpiration = maintenant + 5 min)
- envoie SMS (Firebase Auth ou mock en dev)
- retourne { message: "Code envoyé" }

POST /api/auth/otp/verifier

- reçoit { numeroTel, code }
- vérifie dans codeOTP :
 - code correct ? ☒
 - EstUtilise = false ? ☒
 - dateExpiration > maintenant ? ☒
- marque EstUtilise = true
- crée acheteur si nouveau numéro
- retourne { numeroTel, valide: true }

POST /api/auth/organisateur/inscription → reçoit { nom, numTel, email, motsDePasse } →

hashage motsDePasse avec bcrypt → INSERT dans organisateur → statut = EN_ATTENTE
(validation admin requise) → retourne { message: "Demande envoyée" } POST

/api/auth/organisateur/connexion → reçoit { email, motsDePasse } → vérifie email dans
organisateur → compare motsDePasse avec bcrypt → vérifie statut = VALIDE → retourne JWT
signé POST /api/auth/admin/connexion → même logique que organisateur → vérifie dans
table administrateur POST /api/auth/controleur/connexion → reçoit { codeAcces } → vérifie
dans table controleur → vérifie statut = ACTIF → retourne JWT avec idEvenement inclus

Middleware & sécurité

Middleware authMiddleware.js :

- vérifie présence du token JWT
dans le header Authorization
- decode et vérifie signature
- attache user à req.user
- retourne 401 si token invalide/absent

Middleware roleMiddleware.js :

→ vérifie req.user.role
→ ORGANISATEUR, ADMIN, CONTROLEUR
→ retourne 403 si rôle insuffisant

Appliquer sur toutes les routes privées :

→ /api/evenements → ORGANISATEUR
→ /api/admin/... → ADMIN
→ /api/scan/... → CONTROLEUR

Tests & documentation

Tester dans Postman :

- ☐ Envoi OTP → réception SMS
- ☐ Vérification OTP correct
- ☐ Vérification OTP expiré → erreur
- ☐ Vérification OTP déjà utilisé → erreur
- ☐ Inscription organisateur
- ☐ Connexion organisateur → JWT reçu
- ☐ Route protégée avec JWT valide → accès
- ☐ Route protégée sans JWT → 401
- ☐ Route protégée mauvais rôle → 403
- ☐ Connexion contrôleur avec codeAcces

Écrire un fichier API.md avec

toutes les routes, paramètres et réponses

→ les 2 autres personnes en auront besoin

Personne 2 — Frontend Web React.js

Setup

npx create-react-app frontend-web

npm install axios react-router-dom

react-hook-form tailwindcss

Structure :

frontend-web/src/

```
├── pages/
│   ├── auth/
│   │   ├── ConnexionOrganisateur.jsx
│   │   ├── InscriptionOrganisateur.jsx
│   │   └── ConnexionAdmin.jsx
│   └── components/
│       ├── InputField.jsx
│       ├── Button.jsx
│       └── AlertMessage.jsx
├── services/
│   └── authService.js → appels API
├── context/
│   └── AuthContext.jsx → état global JWT
├── utils/
│   └── storage.js → gestion localStorage JWT
```

Page inscription organisateur

Formulaire avec react-hook-form :

- ☐ Champ nom (requis, min 2 caractères)
- ☐ Champ numéro de téléphone (requis, format +221)
- ☐ Champ email (requis, format email valide)
- ☐ Champ mot de passe (requis, min 8 caractères)
- ☐ Champ confirmer mot de passe (doit correspondre)
- ☐ Bouton soumettre

Au submit :

- POST /api/auth/organisateur/inscription
- Succès → message "Demande soumise, en attente de validation"
- Erreur → afficher message d'erreur

Page connexion organisateur

Formulaire :

- ☐ Champ email
- ☐ Champ mot de passe
- ☐ Bouton connexion

Au submit :

- POST /api/auth/organisateur/connexion
- Succès → stocker JWT dans localStorage
→ rediriger vers /dashboard
- Erreur → "Email ou mot de passe incorrect"
- Statut EN_ATTENTE → "Compte en attente de validation"

Page connexion admin & contexte auth

Page connexion admin :

- Même structure que organisateur
- Appelle /api/auth/admin/connexion
- Redirige vers /admin/dashboard

AuthContext.jsx :

- Stocker user + token JWT
- Fonction login() / logout()
- Vérifier token au chargement de l'app
- Exposer isAuthenticated, userRole

ProtectedRoute.jsx :

- Composant qui vérifie AuthContext
- Si non connecté → redirect /connexion
- Si mauvais rôle → redirect /unauthorized

Page en attente de validation

Écran affiché après inscription organisateur :

- Message clair : "Votre demande est en cours de traitement. Vous recevrez un SMS de confirmation sous 24h."
- Bouton retour à l'accueil

Page unauthorized :

- Affiché si accès refusé
- Message explicatif + bouton retour

Tester l'intégration avec le backend :

- Inscription → message succès
- Connexion → JWT stocké → redirection
- Accès route protégée → OK
- Déconnexion → JWT supprimé → redirect

Responsive & polish

- Vérifier rendu mobile (responsive)
- Ajouter indicateur de chargement sur les boutons lors des appels API
- Gérer les erreurs réseau (pas de connexion internet)
- Tester sur différentes tailles d'écran

Personne 3 — Mobile React Native (Expo)

npx create-expo-app mobile

npm install axios @react-navigation/native

@react-navigation/stack

expo-camera

@react-native-async-storage/async-storage

Setup

Structure :

mobile/src/

| — **screens/**

| | — **auth/**

| | | — **EntrerNumeroScreen.jsx**

| | | — **EntrerOTPScreen.jsx**

| | | — **ConnexionControleurScreen.jsx**

```
|— components/
|   |— InputTel.jsx
|   |— InputOTP.jsx
|   └─ BoutonPrincipal.jsx
|— services/
|   └─ authService.js
|— navigation/
|   └─ AppNavigator.jsx
└─ context/
    └─ AuthContext.jsx
```

Écran entrer numéro de téléphone

Composant InputTel.jsx :

- Format automatique +221 XX XXX XX XX
- Clavier numérique forcé
- Validation : exactement 9 chiffres après +221

EntrerNumeroScreen.jsx :

- Logo SENGUICHET en haut
- Titre "Entrez votre numéro"
- InputTel
- Bouton "Recevoir le code"

Au submit :

- POST /api/auth/otp/envoyer
- Succès → naviguer vers EntrerOTPScreen
avec { numeroTel } en paramètre
- Erreur → afficher alert

Écran entrer code OTP

EntrerOTPScreen.jsx :

- Message "Code envoyé au +221 XX..."
- 6 champs individuels (1 chiffre chacun)
 - focus automatique sur le suivant
- Compte à rebours 5 minutes
- Bouton "Renvoyer le code" (actif après expiration)
- Bouton "Valider"

Au submit :

- POST /api/auth/otp/verifier

- Succès → stocker numeroTel dans AsyncStorage
 - naviguer vers l'accueil
- Code incorrect → "Code invalide"
- Code expiré → "Code expiré, renvoyez-en un"

Écran connexion contrôleur

ConnexionControleurScreen.jsx :

- Titre "Espace Contrôleur"
- Champ code d'accès événement
- Bouton "Accéder"

Au submit :

- POST /api/auth/controleur/connexion
- Succès → stocker JWT contrôleur
 - naviguer vers ScannerScreen
- Code invalide → "Code incorrect ou expiré"
- Code expiré → "Cet événement est terminé"

Navigation & contexte auth

AppNavigator.jsx :

- Stack non authentifié :
 - EntrerNumero → EntrerOTP
- Stack acheteur authentifié :
 - Accueil → Événements → Billet
- Stack contrôleur :
 - Scanner → Résultat scan

AuthContext.jsx :

- Vérifier AsyncStorage au lancement
- Si numeroTel présent → acheteur connecté
- Si JWT présent → contrôleur connecté
- Fonction logout()

Tests & polish

- ☐ Tester flux OTP complet bout en bout
- ☐ Tester expiration OTP sur téléphone réel
- ☐ Tester connexion contrôleur
- ☐ Vérifier que le clavier numérique s'affiche correctement sur iOS et Android
- ☐ Tester la navigation entre écrans
- ☐ Gérer les erreurs réseau (pas de 4G)
- ☐ Vérifier les animations entre écrans